Research Plan — Pip

Project: Pip: A Lightweight AI Assistant Window with Safe Local Tool Access **Category:** iTECH — Software / Programming **Competition:** iWISE Regional Final 2026, Yogyakarta (29 Oct – 1 Nov 2026) **Participants:** Haidar Ali Fawwaz Nasirodin, Faiq Abiyyu Asyam, Avandreda Giano Bestafa **Country:** Indonesia **Supervising teacher:** Muhammad Hasan, Ph.D.

---

1. Research question

> Can a free-tier cloud AI chat service be given controlled, auditable access to a student's local files using only local components, and how much of the resulting risk surface can a user-controlled permission layer close?

The question has two halves and both must be answered with evidence: **can it be done at all**, and **how safe is it once done**. A working demonstration answers the first. Only adversarial testing answers the second.

Secondary question: which failures in such a system are detectable by automated testing, and which survive a passing test suite?

2. Aim

To build a desktop application that lets a student reuse the AI chat account they already have while giving it access to their own files under a boundary the student controls, and to evaluate that application on three axes — whether it functions, what it costs, and what it stops.

The aim is deliberately not "to make AI cheaper" or "to save tokens". Both were considered and both are too narrow: the first is a vendor's problem, the second turned out to be a single feature.

3. Hypotheses and engineering goal

	Statement	Status
H1 — Capability	A free-tier cloud chat account can reach a local MCP tool server and execute a tool call, with no API key, no local model and no paid plan.	**Supported.** Verified live through a public tunnel on a free account.
H2 — Boundary	A user-controlled permission layer — folder allowlist, path canonicalisation, approval before writes, token authentication, audit log — closes the majority of the risk surface of exposing local tools to a remote model.	**Partially supported.** Path traversal and unauthenticated access were blocked; an approval bypass existed and was found by testing, not by review.
H3 — Efficiency *(a sub-hypothesis about one tool)*	Converting a document to Markdown reduces the token cost of a prompt.	**Rejected.** Conversion makes a prompt 12% *larger*. Page selection is the lever, and it is a property of the architecture rather than the converter: 1,844 -> 686 tokens on a 6-page PDF.
Engineering goal	A document must travel from drag-and-drop to a sent prompt, and a tool call must reach an approved write, in one continuous run on the team's own laptop.	Chain verified link by link; the single continuous run had not been performed at the time of writing.

H1 and H2 concern the project. H3 concerns a single tool and is kept as a sub-hypothesis precisely so that a negative result about the converter cannot be mistaken for a negative result about the project. It is reported as rejected. The rejection is a result, not an embarrassment: it is the finding most likely to save another student a week of work, and it is what produced the measurement method now used under the Cost axis.

4. Background

Three pre-existing components, none written by the team:

- **Model Context Protocol (MCP)** — an open standard for exposing tools to an AI client over a transport the client already speaks. This is what lets a cloud chat service call a process on the user's own machine without a bespoke integration per vendor.
- **Microsoft MarkItDown** — converts PDF, DOCX, PPTX and XLSX to Markdown while preserving heading structure. Chosen over writing a converter because conversion is a solved problem and the interesting problem is the permission boundary.
- **Tauri** — a Rust-backed desktop shell rendering the UI in the system webview. Chosen over Electron for binary size, and because a Rust backend makes process spawning an explicit, reviewable command rather than a blanket plugin permission.

Existing AI desktop clients give an assistant access to files, but the boundary is set by the vendor and the account is normally paid. The gap addressed here is the **student on a free tier** who wants both the access and the control.

5. Methodology

5.1 Build (20–22 October 2026)

Incremental, in three tiers, so each tier is a complete project if time runs out:

1. **Tier 1** — always-on-top window, global hotkey, Pomodoro timer, quick notes.
2. **Tier 2** — drag-and-drop document, local conversion to Markdown, editable preview, page selection, token counter.
3. **Tier 3** — local MCP server exposing seven tools (`list_files`, `read_document_as_markdown`, `create_note`, `search_notes`, `save_digest`, `save_quiz`, `reveal_secret`), folder allowlist, approval prompt before any write, append-only audit log.

The riskiest unknown — whether a free account can reach a local MCP server at all — was tested on day 2, before Tier 2 was started. A project that fails H1 is a different project.

5.2 Evaluation, axis 1 — Function

Question: does the chain run end to end?

Method: decompose the chain into seven links and record each as pass, fail, or untested. A link counts as *pass* only when observed in the running application, never when it merely compiles.

#	Link
1	Drag a file onto the Docs tab
2	The app spawns the converter process
3	MarkItDown returns a JSON envelope
4	The preview renders
5	Page selection updates the token counter
6	The AI calls a tool and receives a result
7	A write is blocked, approved, executed and logged

Reporting: a seven-row table plus a single continuous run performed in front of a witness.

Scope decision. The chain originally ended with the prompt written directly into the chat input by a page script. That link was removed on 24 October, and nothing replaced it inside the application: it depended on the chat vendor's DOM and could not be verified without a live account, so a failure on stage was more likely than a success. The preview is a plain editable text box, so the user selects the text and copies it with the operating system. The trade-off is one manual paste and it is stated in the report rather than hidden: **Pip prepares the prompt, it does not deliver it.**

5.3 Evaluation, axis 2 — Cost

Question: what does Pip cost a student per month, compared with the alternative?

Method: measure marginal cost, not sticker price. Three components are recorded separately: subscription delta (the account already exists — is a higher tier required?), tunnel tier (free tier limits), and hardware (an existing laptop, or a purchase?).

The comparison arm is the same ten documents sent through a paid API at list price on the day of measurement. That figure is looked up, not estimated.

Sub-metric — efficiency. Token count per document, measured three ways: raw paste, converted Markdown, and Markdown after page selection. Ten-document corpus covering PDF with a text layer, scanned PDF, DOCX, PPTX, XLSX and text-heavy PDFs. Conversion time recorded as the median of three runs. Percentages are reported per document, never as a corpus average, because the saving depends on how many pages a reader actually needs.

5.4 Evaluation, axis 3 — Safety

Question: what does the permission layer actually stop?

Method: adversarial testing. Each case is an attempt to do something the design says is impossible. A case passes only if the attempt is refused *and* the refusal is logged.

#	Attack or failure mode	Expected result
1	Relative traversal — `../../etc/passwd`	refused
2	Absolute path outside allowlist — `/etc/passwd`	refused
3	Request with no `Authorization` header, empty configured token	refused
4	Unauthenticated request through the public tunnel	refused
5	Read a folder that was never added to the allowlist	refused
6	Write without an approval decision	held pending, not executed
7	Symlink inside an allowed folder pointing outside it	refused
8	A note the model wrote, read back as an unfenced instruction	fenced as data
9	Folder swapped between validation and conversion (TOCTOU)	**known open**

Case 3 is the one that failed on first run: an empty token compared equal to a missing header, so a request with no credentials returned HTTP 200 and a tool result. It was reproduced, fixed, and the same bypass was then found live on the main branch. That sequence — a green test suite with a live bypass — is the answer to the secondary research question.

Case 8 was found the same way, by reading the code rather than the design. `read_document_as_markdown` fenced its output as untrusted data, but `search_notes` returned raw lines, so a note containing an instruction arrived at the model as ordinary text. Because `create_note` lets the model write into that folder, it could plant text it would later read back. One of four tools had skipped the treatment the other three applied.

5.5 Analysis

Three tables, one per axis. Function and Safety are pass/fail counts; Cost is a three-component breakdown plus one comparison figure. The only chart is token count per document, three bars each. Where a result is negative it is reported as negative; the token result is the worked example.

6. Materials and apparatus

macOS laptop (Apple silicon); Tauri 2 with a Rust backend; Node.js and TypeScript; Python 3.13 with MarkItDown in an isolated virtual environment; ngrok and cloudflared; a free Claude account; ten test documents drawn from the team's own coursework.

7. Timeline

Date	Work
20 Oct	Tier 1 — window, hotkey, chat window
21 Oct	MarkItDown spike; tunnel and remote MCP proven (H1); Pomodoro
22 Oct	Tiers 2 and 3 built; review; token and folder-access fixes
23 Oct	Test suite green; packaging phase 0; safety cases 1–6
24–25 Oct	Function run in one pass; cost and efficiency measurement; safety cases 7–8
26 Oct	Report and poster
27 Oct	Print run
28 Oct	Rehearsal and packing
29 Oct – 1 Nov	Exhibition

8. Risks and mitigation

Risk	Likelihood	Mitigation
Venue wifi unavailable -> H1 cannot be demonstrated live	Medium	Declared as a limitation; screen recording made before travel; audit log shows real earlier calls
App is not distributable; demo depends on the developer's checkout	High	Demo runs from the repository on the team's own laptop; packaging listed as future work
Windows path handling untested	High	Scope limited to macOS and stated explicitly
No user study conducted	Certain	Acknowledged as a limitation; the survey instrument is included and administration is future work
TOCTOU between validation and conversion	Low	Documented as open; needs OS-level isolation, out of scope for a seven-day build

9. Ethics and data handling

All test documents are the team's own coursework. No third-party personal data is processed. The application shares no folder by default; access is granted explicitly and every tool call is logged. If the pilot survey is administered later, participation will be voluntary and anonymous and no document content will leave the participant's machine.

10. References

1. Anthropic. *Model Context Protocol specification.* modelcontextprotocol.io
2. Microsoft. *MarkItDown: convert documents to Markdown.* github.com/microsoft/markitdown
3. Tauri Contributors. *Tauri 2 documentation.* tauri.app
4. Model Context Protocol. *Authorization specification.* modelcontextprotocol.io